

Esta práctica es un ejercicio clásico de **Procesamiento de Lenguaje Natural (NLP)**.

El objetivo es construir un clasificador que, al leer el texto de una noticia, sea capaz de predecir a qué categoría pertenece (ej. deportes, política, etc.).

En primer lugar, vamos a visitar la página de Kaggle.com, desde donde descargaremos un set de datos para trabajar, en concreto "Spanish News Classification".

<https://www.kaggle.com/datasets/kevinmorgado/spanish-news-classification>

1. **Proyecto base:** Entrenas un modelo con el texto tal cual.
2. **Iteración 1 (Stemming):** Limpiar el texto reduciéndolo a raíces para ver si la precisión aumenta.
3. **Iteración 2 (Lematización):** Usar análisis lingüístico profundo para ver si mejora aún más.

---

## Bloque 1: Instalación y Carga de Datos

```
# Instala/actualiza la librería de Machine Learning
pip install -U scikit-learn
```

```
# Librería para manejar tablas de datos (DataFrames)
import pandas as pd
```

```
# Carga el archivo CSV
df = pd.read_csv('nombre.csv', encoding='UTF-8')
df.head() # Muestra las primeras filas
```

```
# Vemos un ejemplo de noticia sin procesar
print(f"Noticia original: {df['news'][3]}")
```

- **¿Qué hace?** Lee el archivo CSV y nos muestra el "balanceo" de los datos (cuántas noticias hay de cada categoría). Si hay muy pocas de un tipo, el modelo no aprenderá bien.
-

## Bloque 2: Preparación y División de Datos

```
from sklearn.model_selection import train_test_split

X = df['news'] # Variable independiente (el texto de la noticia)
y = df['type'] # Variable dependiente (la etiqueta o categoría)

X_train, X_test, y_train, y_test = train_test_split(X, y,
test_size=0.2, random_state=42)
```

- **train\_test\_split:** Es fundamental. Divide tus datos en dos grupos:
    1. **Entrenamiento (80%):** Con esto el modelo aprende.
    2. **Prueba (20%):** Datos que el modelo nunca ha visto, para evaluar si realmente aprendió o solo memorizó.
- 

## Bloque 3: Vectorización (Convertir texto a números)

Las computadoras no entienden palabras, solo números.

```
from sklearn.feature_extraction.text import CountVectorizer

vectorizer = CountVectorizer()
X_train_transformed = vectorizer.fit_transform(X_train)
X_test_transformed = vectorizer.transform(X_test)

#Vemos

X_train_transformed #te mantiene las filas de datos, pero tiene un
vector de n columnas

X_train_transformed_dense = X_train_transformed.toarray()

print(X_train_transformed_dense)
```

- **CountVectorizer:** Crea una "bolsa de palabras" (Bag of Words). Cuenta cuántas veces aparece cada palabra en cada noticia.
  - **fit\_transform:** Aprende el vocabulario y transforma el texto en una matriz de números.
  - **X\_train\_transformed\_dense:** Lo que ves aquí es una matriz donde cada columna es una palabra única y cada fila es una noticia.
-

## Bloque 4: Creación del Modelo y Entrenamiento

```
from sklearn.naive_bayes import MultinomialNB
from sklearn import metrics

model = MultinomialNB() # Algoritmo basado en probabilidad, excelente
para texto
model.fit(X_train_transformed, y_train) # ¡Aquí ocurre el aprendizaje!

y_pred = model.predict(X_test_transformed) # El modelo intenta
adivinar las etiquetas del test
print(metrics.accuracy_score(y_test, y_pred)) # Comparamos aciertos vs
realidad
```

- **MultinomialNB:** Es el modelo Naive Bayes. Calcula: "¿Cuál es la probabilidad de que esta noticia sea de 'Deportes' dado que aparecen las palabras 'gol' y 'estadio'?".

El modelo multinomial, o regresión logística multinomial, es una generalización de la regresión logística utilizada para predecir variables dependientes categóricas con más de dos categorías no ordenadas (nominales). Modela la probabilidad de que una observación pertenezca a un grupo específico basándose en variables independientes, comparando cada categoría con una de referencia.

---

## Bloque 5: Mejora con Stemming (Raíces de palabras)

A veces, "correr", "corriendo" y "corrió" deben tratarse como la misma palabra.

```
import nltk
from nltk.tokenize import word_tokenize
from nltk.stem import SnowballStemmer

nltk.download('punkt')
stemmer = SnowballStemmer('spanish')

# Función para procesar el texto
def tokenize_and_stem(text):
    tokens = word_tokenize(text.lower())
    # Solo nos quedamos con letras (quitamos signos de puntuación) y
    aplicamos stemmer
    stems = [stemmer.stem(token) for token in tokens if
token.isalpha()]
    return ' '.join(stems)
```

```
# Aplicamos la función a todo el dataset
df['news_stemmer'] = df['news'].apply(tokenize_and_stem)

# REPETIMOS EL PROCESO CON LOS NUEVOS DATOS
X = df['news_stemmer']
X_train, X_test, y_train, y_test = train_test_split(X, y,
test_size=0.2, random_state=42)

# IMPORTANTE: Hay que volver a ajustar el vectorizador a las nuevas
palabras (raíces)
X_train_vec = vectorizer.fit_transform(X_train)
X_test_vec = vectorizer.transform(X_test)

model.fit(X_train_vec, y_train)
y_pred = model.predict(X_test_vec)
print(f"Precisión con Stemming: {metrics.accuracy_score(y_test,
y_pred)}")
```

- **¿Qué cambió?:** Ahora el vocabulario es más pequeño y eficiente porque agrupamos palabras similares bajo una misma raíz. Reducir el vocabulario eliminando sufijos. Esto ayuda a que el modelo no se confunda con variaciones gramaticales y sea más ligero.

---

## Bloque 6: Mejora con Lematización (Diccionario)

La **Lematización** es más inteligente: lleva la palabra a su forma de diccionario (ej: "fui" -> "ir", "casas" -> "casa"). A diferencia del Stemming (que corta palabras), la lematización busca la palabra base real (el lema).

```
import spacy

# Cargamos el modelo en español de spacy
nlp = spacy.load('es_core_news_sm')

def lemmatize_text(text):
    doc = nlp(text.lower())
    # Extraemos el lema de cada palabra si es una letra
    lemmas = [token.lemma_ for token in doc if token.is_alpha]
    return ' '.join(lemmas)

# Aplicamos lematización (esto puede tardar unos minutos)
df['news_lemma'] = df['news'].apply(lemmatize_text)
```

```
# REPETIMOS EL PROCESO CON LOS DATOS LEMATIZADOS
X = df['news_lemma']
X_train, X_test, y_train, y_test = train_test_split(X, y,
test_size=0.2, random_state=42)

# Volvemos a transformar los datos
X_train_vec = vectorizer.fit_transform(X_train)
X_test_vec = vectorizer.transform(X_test)

model.fit(X_train_vec, y_train)
y_pred = model.predict(X_test_vec)
print(f"Precisión con Lematización: {metrics.accuracy_score(y_test,
y_pred)}")
```

- **¿Qué cambió?:** Usamos `spacy` para entender gramaticalmente el texto. Es más preciso que el Stemming porque no "corta" palabras al azar, sino que usa reglas del lenguaje español.
-